

INTERNATIONAL UNIVERSITY-HCMC UNIVERSITY
WEB APPLICATION DEVELOPMENT
MODULE CODE: ITCS25IU11
LECTURER: ASSOC. PROF NGUYỄN VĂN SINH



GROUP ASSIGNMENT:
CAR MANAGEMENT SYSTEM

GROUP MEMBERS:

Vũ Huỳnh Quốc Anh	ITITWE22003
Vũ Quốc Bảo	ITITWE22107
Phạm Thành Trung	ITITWE22170

Git Link: https://github.com/QuocAnhDiEm/WebApp_project.git

Table of Contents

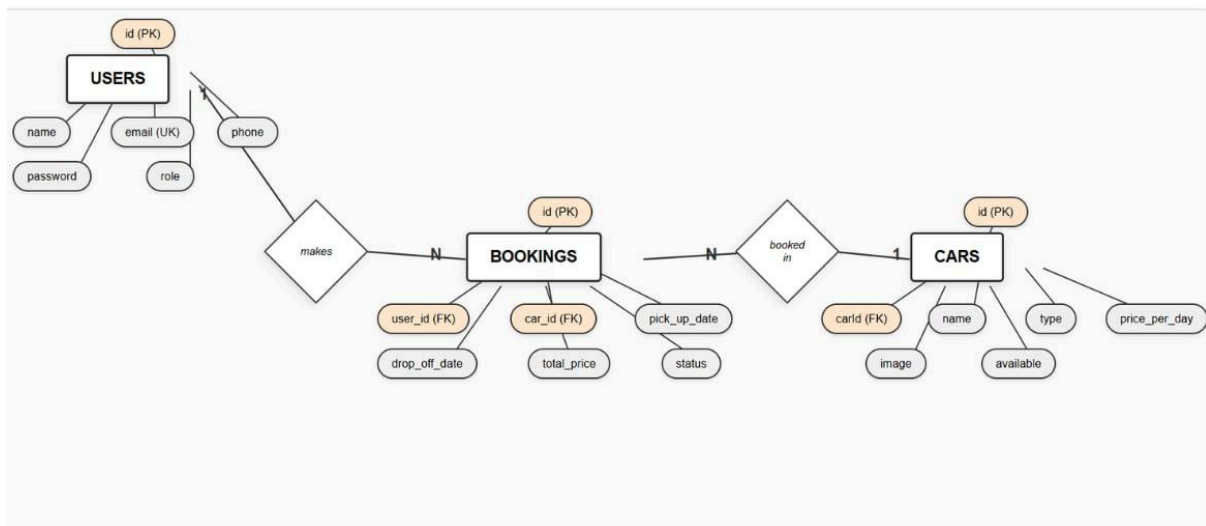
Introduction about our car management system.....	2
1. Core Entities and Attributes.....	2
2. Database Relationships.....	3
I. Home.....	5
My profile.....	6
II. Car Module.....	6
III. Login And Sign-Up.....	7
IV. Book Car and Booking	
Form.....	8 V.
Payment.....	10
VI. Booking Confirmation – Backend Code Explanation.....	11
VII. Dashboard.....	11
VIII. Controller.....	13
8.1 Controller in Car Rental Backend System.....	
13 8.2 Role of the Controller.....	13
8.3 Input Validation and Request Handling.....	
13 8.4 Coordination with Service Layer.....	
13 8.5 Response Management.....	13
8.6 Security Enforcement.....	13
IX. Entities.....	14
9.1 Car Entity.....	
14 9.2 User Entity.....	14
9.3 Booking Entity.....	
14 9.4 Payment Entity.....	15
9.5 Relationship Between Entities.....	15
X. Repository, DTO, and	
Service.....	15
10.1 Repository Layer	
(Data Access).....	15
10.2 DTO (Data	
Transfer Object).....	15
10.3 Service	
Layer (Business Logic).....	16
XI.	
Implementation of Search and Filter Functionality in a Car Rental	
Management System.....	17
11.1 Search by Name or Brand.....	17
11.2 Brand Filter.....	17
11.3 Seat Capacity Filter.....	18
11.4 Filter Button.....	19
XII. Expand about our Controller and How we apply in system.....	20
XIII. How we design in a part for user to change the Password.....	23
XIV. Conclusion.....	

Introduction about our car management system

The Car Management System is a full-stack web-based system that is created to simplify the process of vehicle rental by means of a user-friendly and modern digital platform. The system is developed on the platform of the Spring Boot framework so that the architecture of the system is very robust and these concerns are separated into logical layers such as controllers, services and repositories to achieve the high scalability and data integrity. The site is user-friendly and provides the user with the ability to navigate through the car inventories and filter the data according to particular features such as the brand or the number of seats and operate their profiles in a secure setting secured by Spring Security. A professional Dark Mode design and effective backend logic will help the system to provide a smooth experience to customers and administrators to effectively work on car rental lifecycle.

The Entity-Relationship Diagram (ERD) for your Car Rental Database illustrates the logical structure of the system's data and the relationships between its three core components: Users, Bookings, and Cars.

 **Car Rental Database ERD**



1. Core Entities and Attributes

The system is built around three primary tables, each with specific attributes:

- **USERS Table:** Stores personal information for account management.

.Primary Key (PK): id.

Attributes: name, email (Unique Key), password (hashed for security), phone, and role (e.g., ROLE_USER).

- CARS Table: Contains the technical and pricing details of the vehicle fleet.
.Primary Key (PK): id.

Attributes: name (e.g., Audi R7), type, price_per_day, available (boolean status), and image path for the UI.

- BOOKINGS Table: Acts as the transaction record between a user and a car.
.Primary Key (PK): id.

.Attributes: pick_up_date, drop_off_date, total_price, and status.

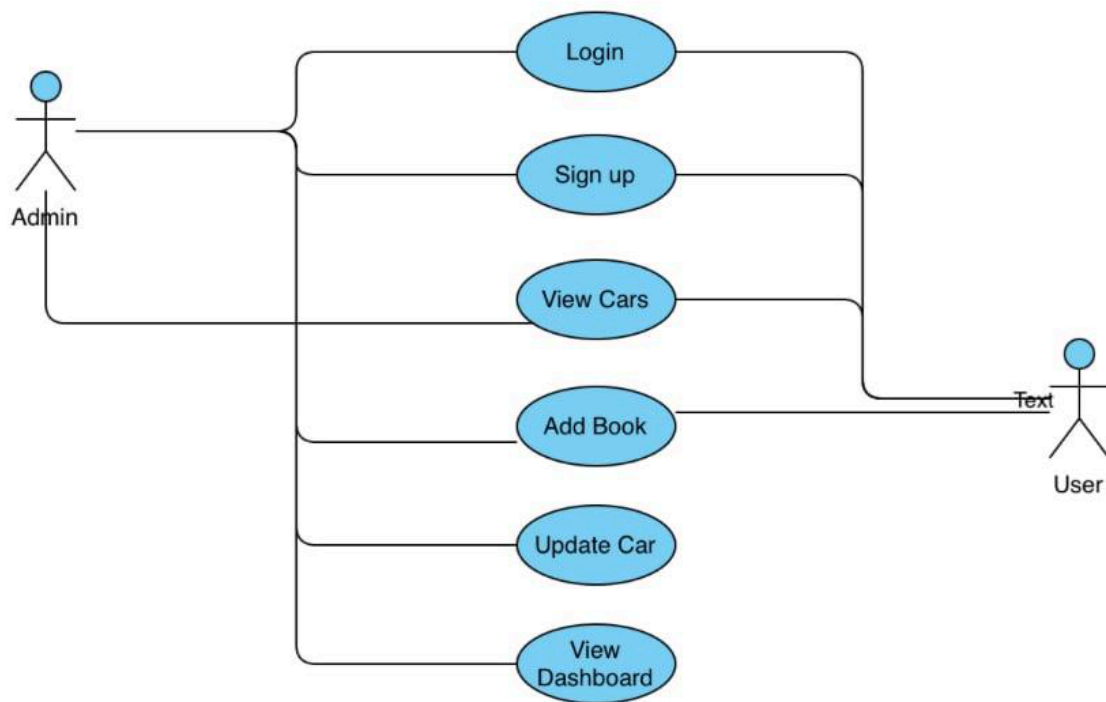
2. Database Relationships

The diagram defines two critical One-to-Many (1:N) relationships, which facilitate the car rental workflow:

Relationship	Type	Description
Users to Bookings	1 : N	One user can make multiple bookings over time, but each specific booking record belongs to only one user. This is linked via the Foreign Key user_id.
Cars to Bookings	1 : N	One specific car can be "booked in" multiple times (on different dates), while each booking record is tied to exactly one car via the Foreign Key car_id.

Use Case Diagram

The Use Case Diagram illustrates the functional requirements of the Car Management System by defining the interactions between different types of users (Actors) and the system's core features (Use Cases).



Actors

The system identifies two primary actors with distinct levels of access:

- Admin: A high-level user responsible for managing the system's inventory and monitoring overall activity.
- User: A general customer who interacts with the platform to view and book vehicles.

Functional Use Cases

The diagram outlines six key functionalities provided by the application:

- Authentication (Login & Sign up): Both the Admin and User can register for an account and log in to the system to access personalized features.
- View Cars: This is a shared functionality allowing both actors to browse the available vehicle fleet.
- Add Book: This feature is specifically assigned to the User, enabling them to initiate a vehicle rental transaction.

- Update Car: Reserved exclusively for the Admin, this allows for the modification of vehicle details, such as price, availability, or specifications.
- View Dashboard: An Admin-only feature used for high-level management and data overview.

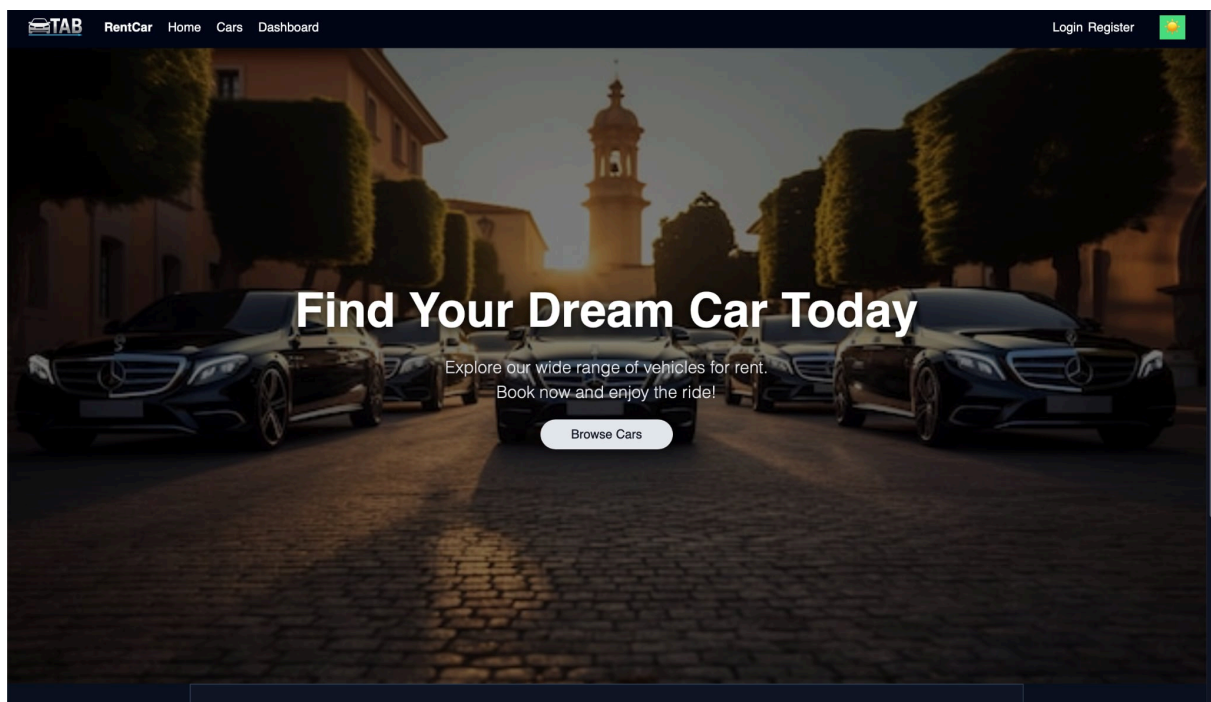
I. Home

The Home module is the main entry point of the Car Rental Management System. It introduces users to the system and provides an overview of available services.

This module presents general information, promotional content, and navigation options to guide users toward other key functionalities.

Main responsibilities of the Home module include:

- Displaying system introduction and service description
- Providing navigation to Car, Login, Contact, Browse Car



My profile

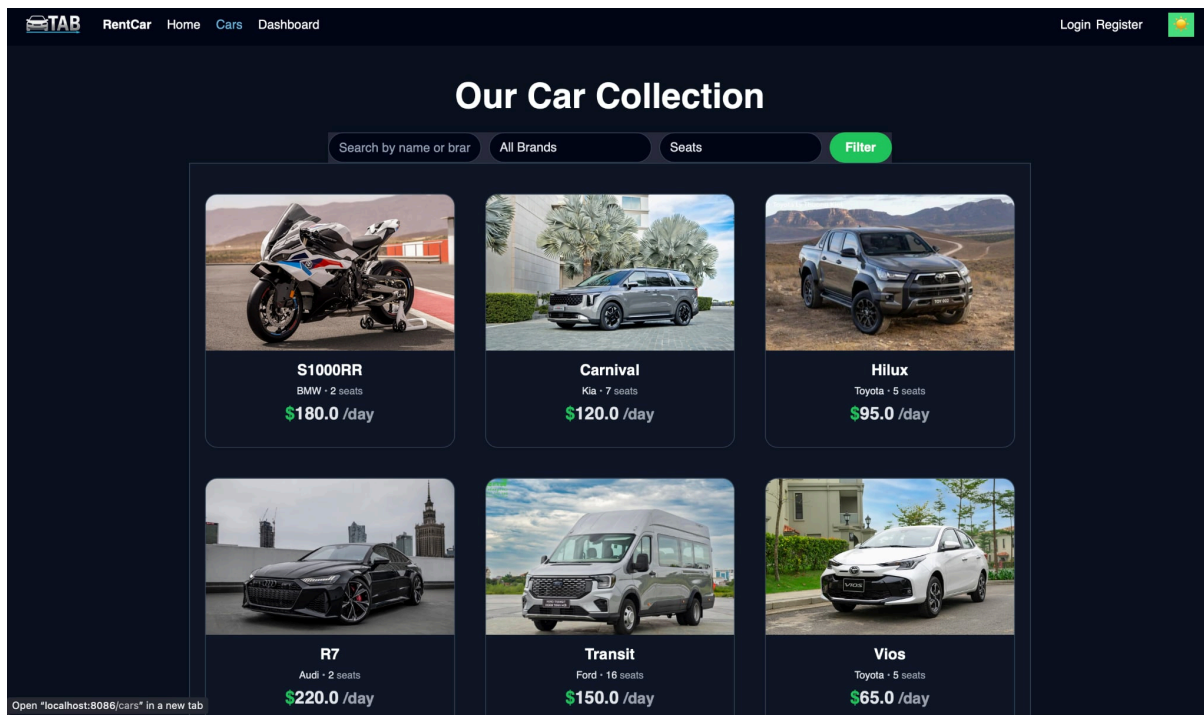
The My Profile page serves as a secure personal dashboard for users. It allows for the management of personal identifiers (Name, Email, Phone) and sensitive credentials through a password update utility. The interface is fully responsive, supporting dynamic theme switching via JavaScript and integrated with Spring Security to ensure that only authenticated users can access and modify their private data.

II. Car Module

The Car module manages all information related to vehicles available for rent. It allows users to browse, view, and compare cars before making a booking decision.

Key functionalities include:

- Displaying a list of available cars
- Showing detailed car information such as model, price, and availability
- Supporting filtering and sorting options



III. Login And Sign-Up

The Login module allows registered users to access the system securely.

Users must provide valid credentials to authenticate their identity.

Main features:

- Secure user authentication
- Error handling for invalid credentials
- Access control to protected system features

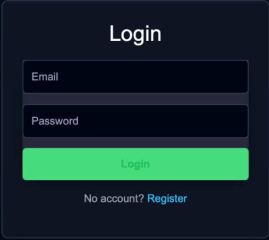
This module plays a critical role in protecting user data and system resources. The

Sign-Up module enables new users to create an account in the system. It collects personal information and stores it securely in the database.

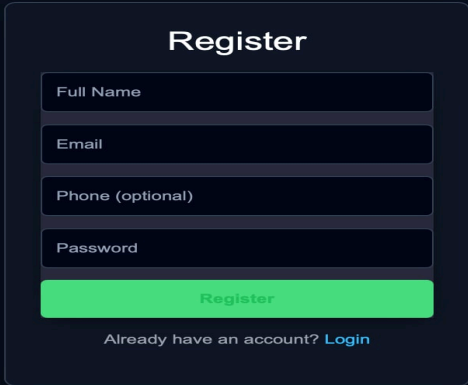
Main functions include:

- User registration form
- Input validation and duplicate account prevention
- Confirmation of successful account creation

This module supports system growth by allowing new users to join easily.



A login form titled "Login" centered on a dark blue background. It features two input fields: "Email" and "Password". Below these fields is a prominent red "Login" button. At the bottom of the form, there is a link that says "No account? Register".



A registration form titled "Register" centered on a dark blue background. It contains four input fields: "Full Name", "Email", "Phone (optional)", and "Password". Below these fields is a prominent red "Register" button. At the bottom of the form, there is a link that says "Already have an account? Login".

IV. Book Car and Booking Form

The Book Car module allows users to reserve a selected vehicle. It connects directly with the Car and Payment modules.

This module includes:

- Booking form for selecting rental dates
- Automatic cost calculation based on rental duration
- Availability verification before confirmation

Accurate data collection in this module helps prevent booking conflicts.

After the user submits the booking form, the form data is sent to the backend booking controller.

This data includes the selected car, rental dates, pickup location, and user information.

The backend first performs input validation to ensure that all required fields are provided and that the rental period is valid.

After validation, the system checks the availability of the selected car to avoid overlapping bookings.

If the car is available, a booking record is created with a **pending** status. At this stage, the booking is not confirmed yet and awaits payment processing.

The screenshot shows a 'Book Your Car' form on a dark-themed interface. At the top left is a link '← Back to Cars'. The form is titled 'Book Your Car' and contains the following sections:

- Car Selection:** A box showing 'Carnival' in green, 'MPV' below it, and '120.0 \$/day' in bold.
- Personal Information:** Fields for 'First Name *' (containing 'John') and 'Last Name *' (containing 'Doe').
- Contact Information:** Fields for 'Email Address' (containing 'john.doe@example.com') and 'Phone Number *' (containing '+1 (555) 123-4567').
- Rental Details:** Fields for 'Pick-up Location *' (placeholder 'Enter pick-up location'), 'Drop-off Location *' (placeholder 'Enter drop-off location'), 'Pick-up Date *' (containing '12/27/2025'), and 'Drop-off Date *' (containing '12/27/2025').
- Additional Notes (Optional):** A text area with the placeholder 'Any special requests or requirements?'.

A green 'Confirm Booking' button is at the bottom of the form.

V. Payment

The Payment module handles all financial transactions within the system. It ensures that payments are processed securely and accurately.

Key responsibilities include:

- Displaying total rental cost
- Processing online payments
- Confirming successful transactions

This module enhances user convenience and transaction reliability.

Once the booking form is successfully processed, the system moves to payment handling.

The backend receives the selected payment method and payment details from the user.

The system calculates the total rental cost based on the rental duration and the price of the selected car.

After that, the backend verifies the payment information and processes the transaction.

If the payment is successful, the booking status is updated from **pending** to **confirmed**.

The confirmed booking is then stored in the database and a confirmation response is returned to the user.

Payment Method

Booking Summary

Car Toyota Hilux	Price/Day \$95.0	Pick-up Date 2025-12-25	Drop-off Date 2025-12-26
Customer Thành Trung Phạm	Phone +84394674592	Pick-up Location Ba	Drop-off Location Ria
Email ttrung1326@gmail.com	Total Days 1	Total Price \$95.0	

Additional Notes

Cash Payment

You will pay in cash when you pick up the vehicle at our location. Please bring the exact amount or we can provide change.

Important Information

- Payment must be made in cash upon vehicle pick-up
- Please bring a valid driver's license
- A security deposit may be required

Confirm Booking

VI. Booking Confirmation – Backend Code Explanation

After the user submits the booking form, the request is sent to the backend booking controller.

The controller first extracts booking information such as car ID, rental dates, and user ID from the request.

Next, the system calls the car service to check whether the selected car is available during the requested period.

This is done by querying the booking data and validating that there is no overlapping reservation.

If the car is available, the backend creates a new booking object and calculates the total rental cost based on the rental duration and car price.

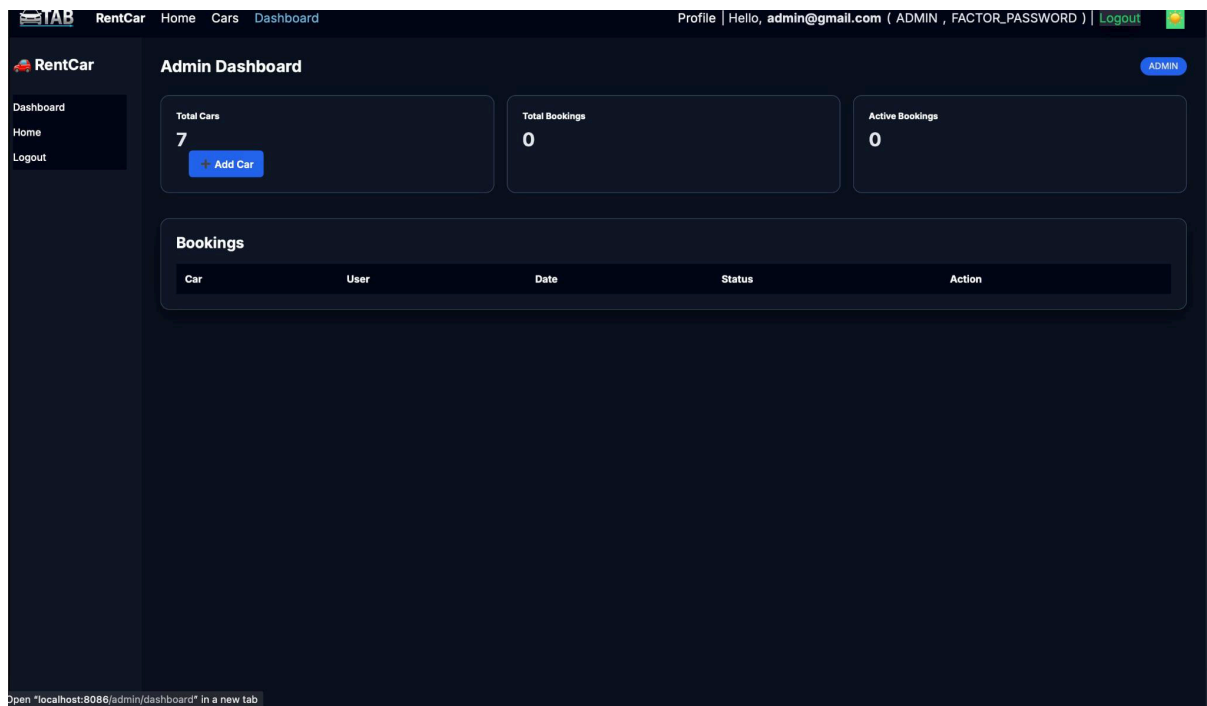
The booking data is then stored in the database with a status set to **“confirmed”**.

Finally, the backend returns a booking confirmation response containing the booking ID, car details, rental period, and total price.

This response is used to display the booking confirmation to the user or trigger further actions such as payment processing.

VII. Dashboard

The Admin Dashboard serves as the central management hub for the car rental system, providing administrators with real-time oversight of the platform's assets and user activities.



- **Platform Overview:** The dashboard displays key performance indicators at a glance, including Total Cars currently in the fleet, the count of Total Bookings, and the number of Active Bookings currently in progress.
- **Booking Management (View/Delete):** Administrators can monitor all transactions in the Bookings Table. This feature allows them to view detailed information for each reservation—such as the car assigned, the associated user, the booking date, and the current status—and provides the necessary administrative actions to remove or manage these bookings as needed.
- **Fleet Management (Add Car):** To ensure the rental service can scale, the dashboard includes a direct "Add Car" functionality. This allows administrators to immediately register new vehicles into the system database, making them available for user search and booking.
- **User Context:** The system maintains administrative security by displaying the logged-in admin's profile and roles (e.g., ADMIN, FACTOR_PASSWORD) in the top navigation bar, ensuring that only authorized personnel can access these management tools.

VIII. Controller

8.1 Controller in Car Rental Backend System

The controller acts as the main entry point for all client requests in the Car Rental system.

It is responsible for receiving requests from the booking form, validating input data, and coordinating interactions between different backend components.

8.2 Role of the Controller

When a user submits a booking or selects a payment method, the request is first handled by the controller.

The controller extracts necessary information such as user ID, car ID, rental dates, and payment details from the request.

8.3 Input Validation and Request Handling

Before passing data to the business logic, the controller performs basic validation checks.

This includes verifying required fields, checking data formats, and ensuring that the request comes from an authenticated user.

Invalid requests are rejected immediately to protect backend resources.

8.4 Coordination with Service Layer

The controller does not implement business logic directly.

Instead, it delegates tasks to service components such as booking service, car service, or payment service.

This separation of concerns improves maintainability and code organization.

8.5 Response Management

After receiving results from the service layer, the controller formats and returns a response.

For example, it may return booking confirmation details, payment status, or error messages.

8.6 Security Enforcement

The controller enforces security rules by ensuring that only authorized users can access booking and payment endpoints.

It also prevents unauthorized access to sensitive operations such as booking modification or cancellation.



IX. Entities

9.1 Car Entity

The Car entity represents vehicles available for rental.

It contains attributes such as car ID, brand, model, price per day, and availability status.

This entity is used to determine which cars can be booked and to calculate rental costs.

9.2 User Entity

The User entity stores information about registered users.

It includes user ID, name, contact details, and authentication-related data. The backend uses this entity to identify users and control access to booking and payment functions.

9.3 Booking Entity

The Booking entity records rental transactions between users and cars. It contains booking ID, user ID, car ID, rental period, booking status, and total cost. This entity plays a central role in confirming reservations and preventing double bookings.

9.4 Payment Entity

The Payment entity stores information related to payment processing. It includes payment ID, booking ID, payment method, and payment status. Sensitive payment details are not stored directly to ensure system security.

9.5 Relationship Between Entities

Entities are interconnected within the backend system.

A user can create multiple bookings, each booking is linked to one car, and each booking is associated with a payment record.

These relationships ensure data consistency and reliable backend operations.

X. Repository, DTO, and Service

10.1 Repository Layer (Data Access)

The Repository layer is the closest to the database. It uses Spring Data JPA to handle all CRUD (Create, Read, Update, Delete) operations.

- Purpose: To abstract the database logic so the rest of the application doesn't need to write complex SQL queries.
- Implementation: You create interfaces that extend `JpaRepository<Entity, Id>`.
- In your project: For example, a `UserRepository` would use `findByEmail(String email)` to check if a user exists during login.
- Key Benefit: It allows you to perform database actions using simple method names (e.g., `save(user)` as seen in Image 1 and 2).

10.2 DTO (Data Transfer Object)

DTOs are simple Java objects used to pass data between the frontend and the backend.

- Purpose: To hide the internal details of your database entities. For instance, you wouldn't want to send a user's hashed password or internal ID back to the browser in a simple "Profile View" request.
- Usage in your project:
 - `RegistrationDTO`: Contains only email, password, and `confirmPassword`.

- Booking TO: Contains carId, startDate, and endDate.
- Validation: DTOs are often where you put validation annotations like `@NotBlank` or `@Email` to ensure the data coming from the user is correct before it ever reaches your database.

10.3 Service Layer (Business Logic)

The Service layer sits between the Controller and the Repository. It contains the "brain" of your application.

- Purpose: To handle complex business rules that shouldn't be in the Controller.
- Implementation: This consists of an Interface (like `UserService` in Image 2) and an Implementation class (`UserServiceImpl`).
- Logic Example: In your `changePassword` logic (Image 3), the Controller handles the request, but the Service is responsible for:
 1. Encrypting the new password.
 2. Comparing the provided `currentPassword` with the one stored in the database.
 3. Returning a boolean success/fail status back to the Controller.
- Transaction Management: Services ensure that if an error occurs during a multi-step process (like booking a car and charging a card), the whole process "rolls back" so data doesn't get corrupted.

Layer	Responsibility	Key Example in Car Rental Project
Repository	Database Communication	<code>bookingRepository.findAll()</code> to show the list on the Admin Dashboard.
DTO	Data Packaging	<code>CarDTO</code> used to transfer car details from the "Add Car" form to the server.
Service	Business Logic	<code>changePassword()</code> logic to verify credentials before updating the DB.

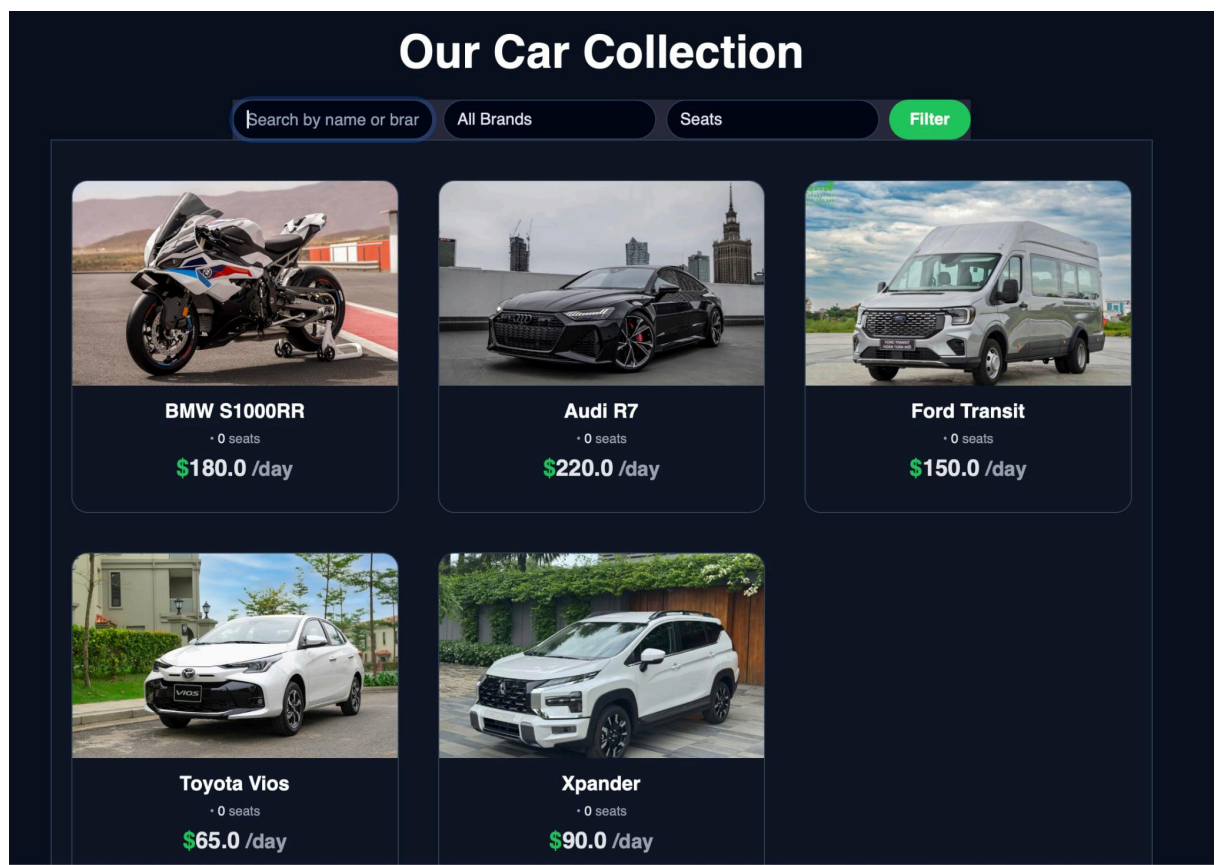
XI. Implementation of Search and Filter Functionality in a Car Rental Management System

The **Search and Filter module** is designed to help users quickly find suitable vehicles from the car collection based on specific criteria. This feature improves usability, efficiency, and user experience in the car rental management system.

11.1 Search by Name or Brand

The search bar allows users to enter keywords such as the **car name** or **brand**.

- The system performs real-time filtering.
- Only vehicles whose names or brands match the entered keyword are displayed.
- This feature reduces browsing time and makes the system more user-friendly.



11.2 Brand Filter

The **Brand dropdown filter** enables users to select a specific manufacturer, such as:

- BMW
- Toyota
- Kia
- Audi
- Ford
- Mitsubishi

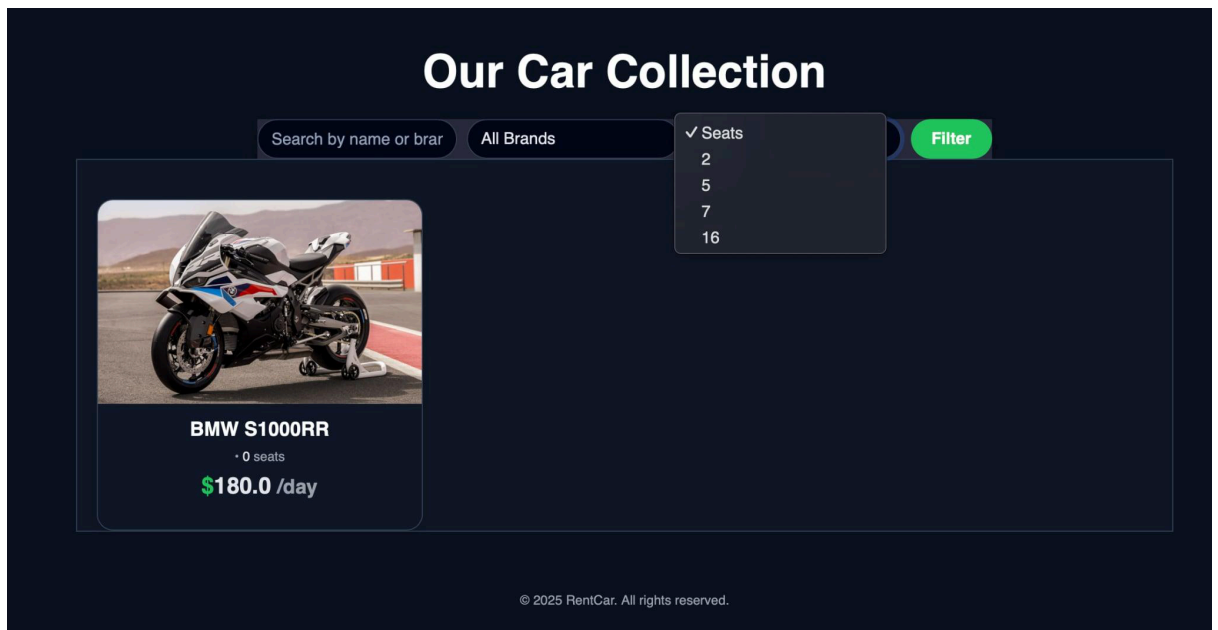
When a brand is selected:

- The system updates the car list dynamically.
- Only vehicles belonging to the chosen brand are shown.
- Selecting “**All Brands**” resets the filter and displays all available vehicles.

11.3 Seat Capacity Filter

The **Seats filter** allows users to sort cars based on seating capacity.

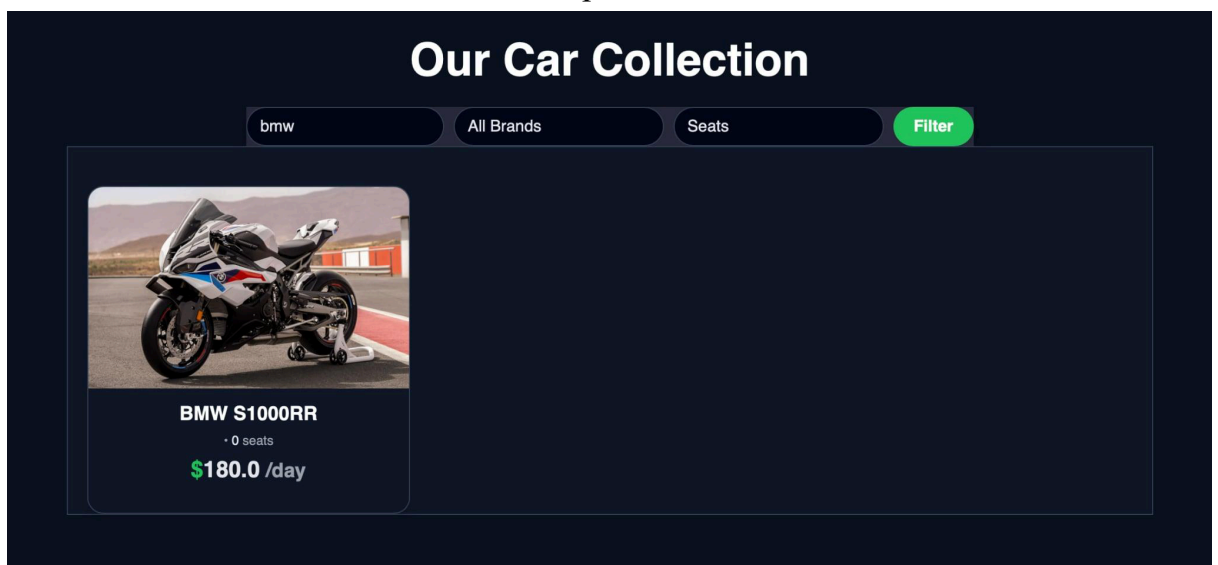
- Available options include: 2, 5, 7, and 16 seats.
- This feature is useful for customers with different travel needs (individuals, families, or groups).
- The system displays only cars that match the selected seat number.



11.4 Filter Button

The **Filter button** applies all selected criteria simultaneously.

- The system combines search text, brand selection, and seat capacity.
- Results are updated instantly without reloading the page.
- This ensures smooth interaction and better performance.



XII. Expand about our Controller and How we apply in system

1. When the user performs a search or sort action, the request is first sent to the CarController, where the search and filtering parameters are processed before retrieving and displaying the corresponding car data:

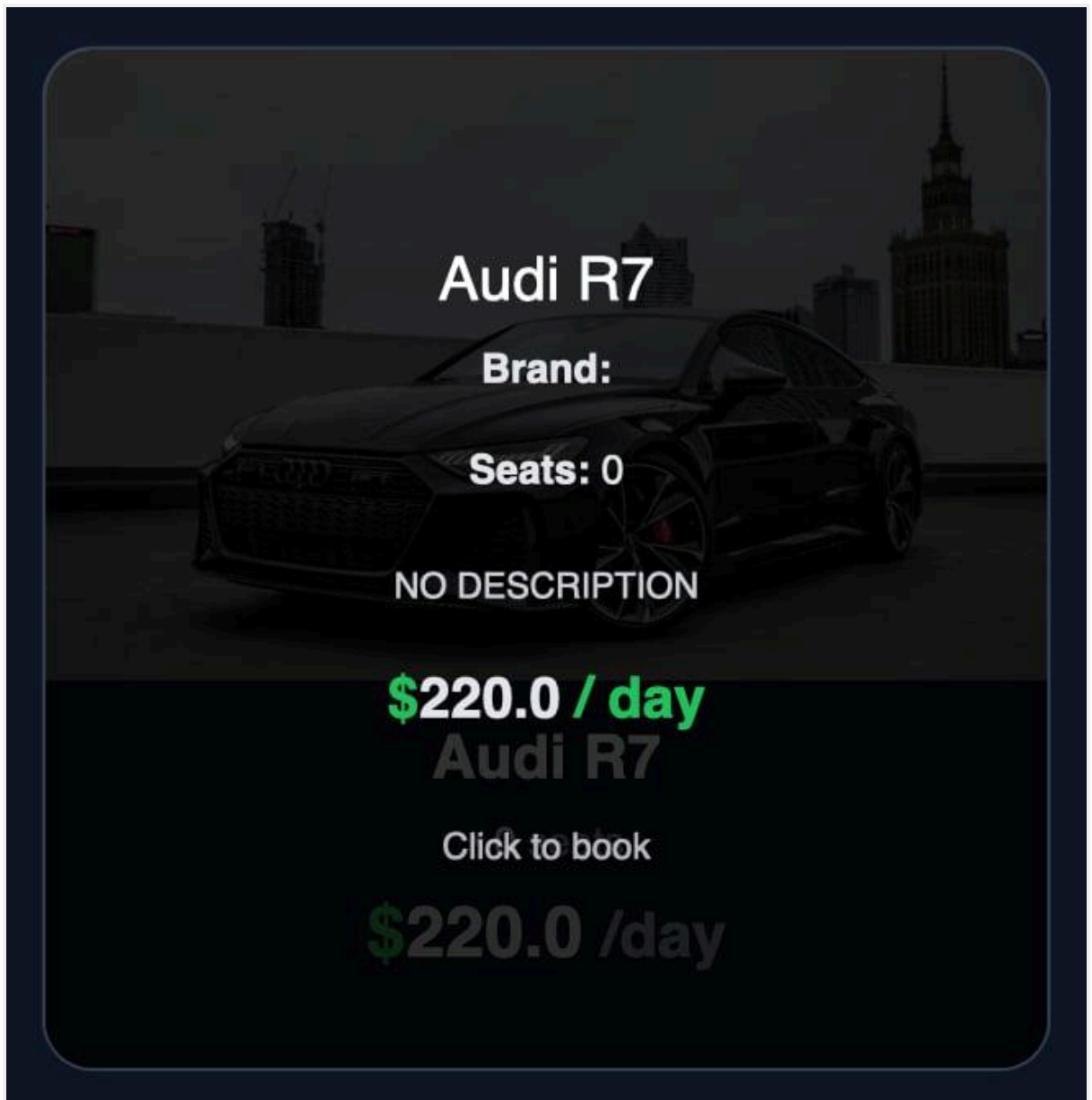
```
http://127.0.0.1:8086/cars (Count=9 Total=0.31s Max=0.00s)
23  @GetMapping("/cars")
24  public String cars(
25      @RequestParam(required = false) String keyword,
26      @RequestParam(required = false) String brand,
27      @RequestParam(required = false) Integer seats,
28      Model model
29  ) {
30      List<Car> cars;
31
32      if (keyword != null && !keyword.isBlank()) {
33          cars = carService.searchCars(keyword);
34      } else if (brand != null && !brand.isBlank()) {
35          cars = carService.filterByBrand(brand);
36      } else if (seats != null) {
37          cars = carService.filterBySeats(seats);
38      } else {
39          cars = carService.getAvailableCars();
40      }
41
42      model.addAttribute("cars", cars);
43      model.addAttribute("keyword", keyword);
44      model.addAttribute("brand", brand);
45      model.addAttribute("seats", seats);
46
47      return "cars"; // templates/cars.html
48  }
```

2. The CarService interface serves as the primary service layer for car management. It facilitates read operations through five key methods: general retrieval, ID-based lookup, keyword-based searching, and attribute-specific filtering (by brand and seat capacity). By defining these operations in an interface, the system ensures a decoupled architecture, allowing for different implementation strategies of the car rental business logic.

```
1 package com.example.car_rent.service;
2
3 import com.example.car_rent.entity.Car;
4 import java.util.List;
5
6 public interface CarService {
7
8     List<Car> getAvailableCars();
9
10    Car getCarById(Long id);
11
12    // 🔍 Search by name or brand
13    List<Car> searchCars(String keyword);
14
15    // 🎯 Filters
16    List<Car> filterByBrand(String brand);
17
18    List<Car> filterBySeats(Integer seats);
19 }
20
```

3. After that all the information transfer to CarRepository->Data->Get back->Give theResult to the User:

```
→ AdminController CarServiceImpl BookingServiceImpl | ← 3 beans
10 public interface CarRepository extends JpaRepository<Car, Long> {
11
12     // Search by name or brand
13     Show AOT-generated Implementation, Query, etc...
14     @Query("""
15         SELECT c FROM Car c
16         WHERE lower(c.name) LIKE lower(concat('%', :keyword, '%'))
17         OR lower(c.brand) LIKE lower(concat('%', :keyword, '%'))
18         """)
19     List<Car> search(@Param("keyword") String keyword);
20
21     // Filter by brand
22     Show AOT-generated Implementation, Query, etc...
23     List<Car> findByBrandIgnoreCase(String brand);
24
25     // Filter by seats
26     Show AOT-generated Implementation, Query, etc...
27     List<Car> findBySeats(int seats);
28
29     // Filter by type
30     Show AOT-generated Implementation, Query, etc...
31     List<Car> findByTypeIgnoreCase(String type);
32
33     // Filter by availability
34     Show AOT-generated Implementation, Query, etc...
35     List<Car> findByAvailableTrue();
36 }
```



XIII.How we design in a part for user to change the Password

The car management project implements a robust user profile and security management system following the Model-View-Controller (MVC) architecture. In the Service Layer (UserService), a contract is established to handle core data operations, including a specific method for password verification and updates. The Controller Layer manages these features through two primary endpoints: /profile/update for general information and /profile/change-password for security credentials. For profile updates, the system retrieves the authenticated user's principal, modifies attributes like name and phone, and persists the changes to the database. The password change logic adds a layer of validation by checking if new passwords match and verifying the current password through the service layer before committing changes. Throughout both processes, RedirectAttributes are utilized to provide real-time, asynchronous

feedback to the user via success or error "flash" messages, ensuring a seamless and secure user experience.

```
58 http://127.0.0.1:8086/profile/update
59 @PostMapping("/profile/update")
60 public String updateProfile(
61     @AuthenticationPrincipal UserPrincipal userPrincipal,
62     @RequestParam String name,
63     @RequestParam String phone,
64     RedirectAttributes redirectAttributes
65 ) {
66     User user = userPrincipal.getUser();
67
68     user.setName(name);
69     user.setPhone(phone);
70
71     userService.save(user);
72
73     redirectAttributes.addFlashAttribute(
74         "success", "Profile updated successfully"
75     );
76     return "redirect:/profile";
77 }
78
79 }
80
```

```
1 package com.example.car_rent.service;
2
3 import com.example.car_rent.entity.User;
4
5 public interface UserService {
6
7     User findByEmail(String email);
8
9     User save(User user);
10
11     // 🗝️ Change password
12     boolean changePassword(User user, String currentPassword, String newPassword);
13 }
14
```

```

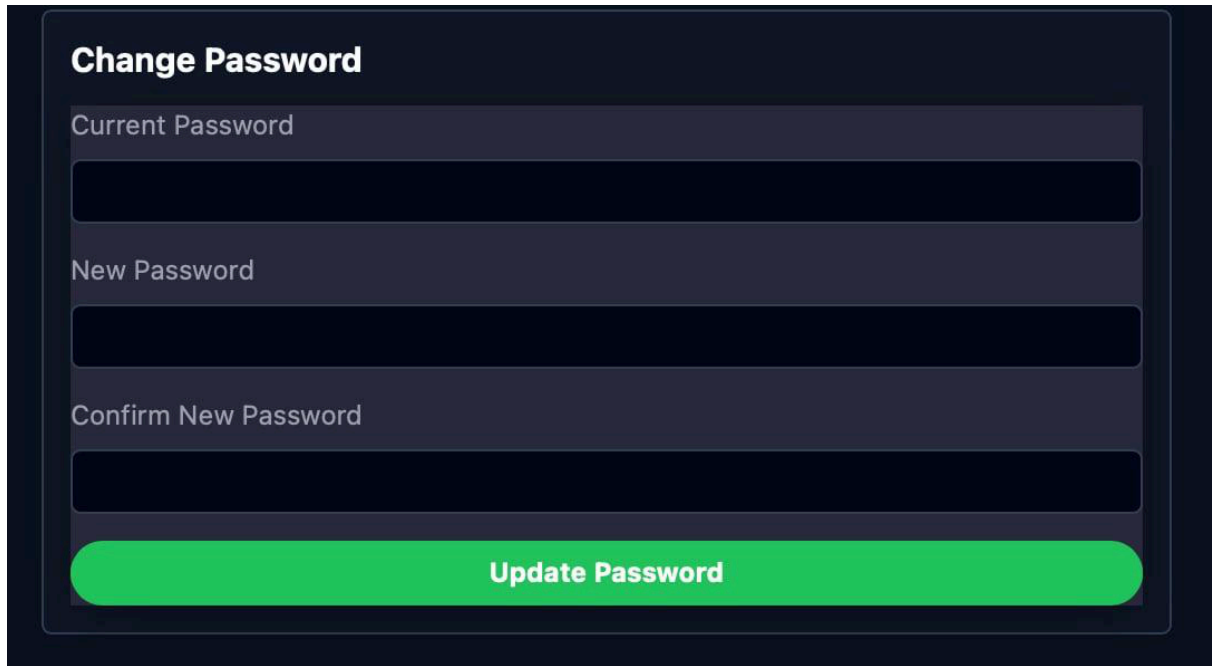
http://127.0.0.1:8086/profile/change-password
30 @PostMapping("/profile/change-password")
31 public String changePassword(
32     @AuthenticationPrincipal UserPrincipal userPrincipal,
33     @RequestParam String currentPassword,
34     @RequestParam String newPassword,
35     @RequestParam String confirmPassword,
36     RedirectAttributes redirectAttributes
37 ) {
38
39     if (!newPassword.equals(confirmPassword)) {
40         redirectAttributes.addFlashAttribute("error", "Passwords do not match");
41         return "redirect:/profile";
42     }
43
44     boolean success = userService.changePassword(
45         userPrincipal.getUser(),
46         currentPassword,
47         newPassword
48     );
49
50     if (!success) {
51         redirectAttributes.addFlashAttribute("error", "Current password is incorrect");
52         return "redirect:/profile";
53     }
54
55     redirectAttributes.addFlashAttribute("success", "Password updated successfully");
56     return "redirect:/profile";
57 }
58

```

Change Password is an option controlled by a controller method that responds to the secure form submissions to the /profile/change-password endpoint. This is done by first ensuring that the new password and password confirmation are the same and in case they are not, the system will automatically go back to the profile page with an error message. In case of matching, the controller makes a call to the userService.changePassword method and provides the authenticated user with details and current and new passwords to verify them. In case the service layer concludes that the present password is wrong, a failure message will be returned, otherwise a password is successfully changed, and a message of success will be shown to the user after a final redirect.

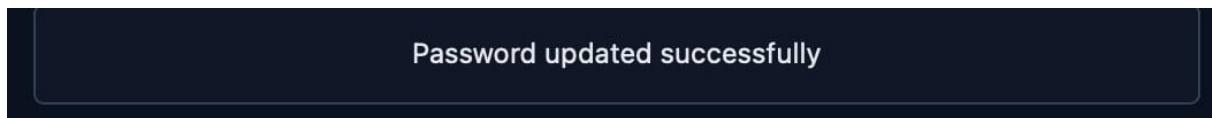
After that these are the results:

Before:



The image shows a 'Change Password' form with a dark background. It contains three input fields labeled 'Current Password', 'New Password', and 'Confirm New Password'. Below these fields is a bright green button with the text 'Update Password'.

After:



The image shows a dark-themed notification box with a light blue border. Inside the box, the text 'Password updated successfully' is displayed in a light blue font.

XIV. Conclusion

In conclusion, this project successfully demonstrates the design and implementation of a backend-oriented Car Rental system. The system effectively manages core functionalities such as car booking, payment processing, and data handling through a well-structured backend architecture.

By applying clear separation of concerns between controllers, services, and entities, the system ensures maintainability, scalability, and secure data processing. Backend validation, authentication, and availability checks play a crucial role in preventing invalid bookings and protecting user information.

Overall, the project highlights the importance of backend logic in building reliable rental systems and reflects a solid understanding of backend design principles relevant to real-world applications and academic study.